Healthcare Data Warehouse & Automated Pipeline Documentation

Team DataX:-
1- Dina Farid Mohamed
2- Pansseh Mahmoud
3- Aya
4- Fatma

1. Executive Summary

This document provides a comprehensive overview of the Healthcare Data Warehouse architecture and its automated ETL (Extract, Transform, Load) pipelines. The system implements a robust **Medallion Architecture** (Staging ➔ Bronze ➔ Silver ➔ Gold) on Microsoft SQL Server. It is governed by a fully automated Python orchestration layer that handles synthetic data generation, database bridging, and SQL stored procedure execution.

Furthermore, the architecture integrates advanced Machine Learning (ML) predictions, Exploratory Data Analysis (EDA), and exposes an optimized Star Schema for a Power BI Semantic Model enriched with DAX KPIs and Time Intelligence.

2. Infrastructure & Database Setup

Script Reference: database_definetion.sql, logging_utilites.sql

2.1 Database Configuration

- **Database Name:** HealthcareDataWarehouse
- **Recovery Model:** Configured to SIMPLE to prevent "Transaction Log Full" errors during massive batch data operations.
- **Schemas:** Organized into five distinct schemas representing the data lifecycle and operations:
  - ops: Pipeline logging and execution tracking.
  - staging: Temporary landing zone for raw flat files.
  - bronze: Historical raw data archive with lineage.
  - silver: Cleansed, typed, and enriched data.
  - gold: Business-ready views configured as a Star Schema.

2.2 Operational Logging Framework (ops schema)

A custom logging utility tracks the performance and status of every pipeline run.

- **Tables:** * ops.job_runs: Tracks high-level pipeline executions (pipeline_name, status,

duration).
    - ops.task_runs: Tracks individual steps within a pipeline (e.g., rows inserted, granular status).
- **Stored Procedures:** sp_start_job, sp_end_job, sp_start_task, sp_end_task
- **Benefit:** Ensures end-to-end observability, making it easy to identify exactly when and where a failure occurred during the nightly batch runs.

3. Python Orchestration & Data Engineering

The pipeline's execution is fully decoupled and managed via Python scripts, ensuring a resilient, automated schedule.

3.1 Data Generation

- Uses Faker and pandas to generate robust, simulated healthcare datasets mirroring real-world complexities (e.g., readmissions, clinical vitals, financial charges, CAHPS satisfaction scores, staff burnout indices).
- Exports output to healthcare_simulation.csv.

3.2 Database Bridge

- Connects the local CSV output to the SQL Server using sqlalchemy and pyodbc.
- Dynamically uploads the raw data directly into the SQL Staging Layer (staging.Healthcare_Dataset).

3.3 Pipeline Runner

- Connects to SQL Server and sequentially triggers the Medallion stored procedures (Staging ➔ Bronze ➔ Silver).
- Provides live terminal feedback and generates an external healthcare_pipeline.log file.
- Automatically queries the ops.job_runs table at the end of the run to display a formatted summary table of execution times.

3.4 Master Scheduler

- Acts as the overarching orchestrator.
- Uses the schedule library to trigger the full ETL sequence automatically at a defined PIPELINE_SCHEDULE_TIME (configured via .env).
- Executes Step 1 (Generation) ➔ Step 2 (Bridge) ➔ Step 3 (SQL Pipelines).

4. The Medallion Architecture (SQL Layer)

Layer 1: Staging (Raw Landing Zone)

- **Purpose:** Acts as a volatile landing area for incoming CSV files.
- **Data Structure:** All columns are VARCHAR(255) to prevent structural ingest failures.

- **Logic (sp_run_healthcare_staging_load):** Truncates existing tables and uses BULK INSERT for rapid data ingestion. Wraps execution in TRY...CATCH with ops logging.

Layer 2: Bronze (Historical Raw & Lineage)

- **Purpose:** Immutable, historical record of all ingested data with metadata (ingested_at, batch_id).
- **Logic (sp_run_healthcare_bronze_load):** Employs a Full Row Hash anti-join (HASHBYTES('SHA2_256', ...)) to guarantee idempotency and prevent duplicate records.

Layer 3: Silver (Cleansed, Integrated & Enriched)

- **Purpose:** Single source of truth. Data is typed, standardized, validated, and enriched.
- **Logic (sp_run_healthcare_silver_load):**
  - **Cleansing:** Standardizes text and formats.
  - **Typing:** Uses TRY_CAST to gracefully handle dirty rows.
  - **Calculations:** Adds flags for SLA breaches, true length of stay, financial write-offs, and logical sequence constraint checks (sequence_status).
  - **ML Merge:** Appends Machine Learning predictions (ML_Results) to the core dataset.

Layer 4: Gold (Business Intelligence Presentation)

- **Purpose:** Denormalized Star Schema views optimized for BI reporting.
- **Dimensions:** vw_Dim_Patient, vw_Dim_Diagnosis, vw_Dim_Nurse, vw_Dim_Doctor, vw_Dim_Payer, vw_Dim_Date_final.
- **Fact:** vw_Fact_Admissions consolidates metrics and calculates on-the-fly "Virtual Bands" (e.g., severity_band, satisfaction_band, ML_Risk_Band).

5. Exploratory Data Analysis (EDA) & Machine Learning

5.1 Exploratory Data Analysis (EDA.py)

- Leverages seaborn and matplotlib to profile the generated data.
- Analyzes birth year distributions, comorbidity frequencies, wait times versus length of stay, staff overtime correlations, top denial reasons, and HEDIS compliance.

5.2 Predictive Machine Learning Pipeline (ML3.py)

Extracts data from the Silver layer in chunked streams (capable of handling 1.1M+ rows) and trains core predictive models:

1. **Safety Incident Prediction (Logistic Regression):** Predicts likelihood of safety incidents using vitals (sbp_mmhg, heart_rate_bpm, wbc_10e3_ul) and operational metrics (wait_time_mins, bed_occupancy_pct).

2. **Length of Stay (LOS) Prediction (Linear Regression):** Forecasts patient stay duration utilizing severity_index, comorbidity_count, age, and assigned unit.
3. **Financial Cost Prediction (Linear Regression):** Predicts total submitted charges based on drg_weight, true_los_days, severity_index, and payer constraints.

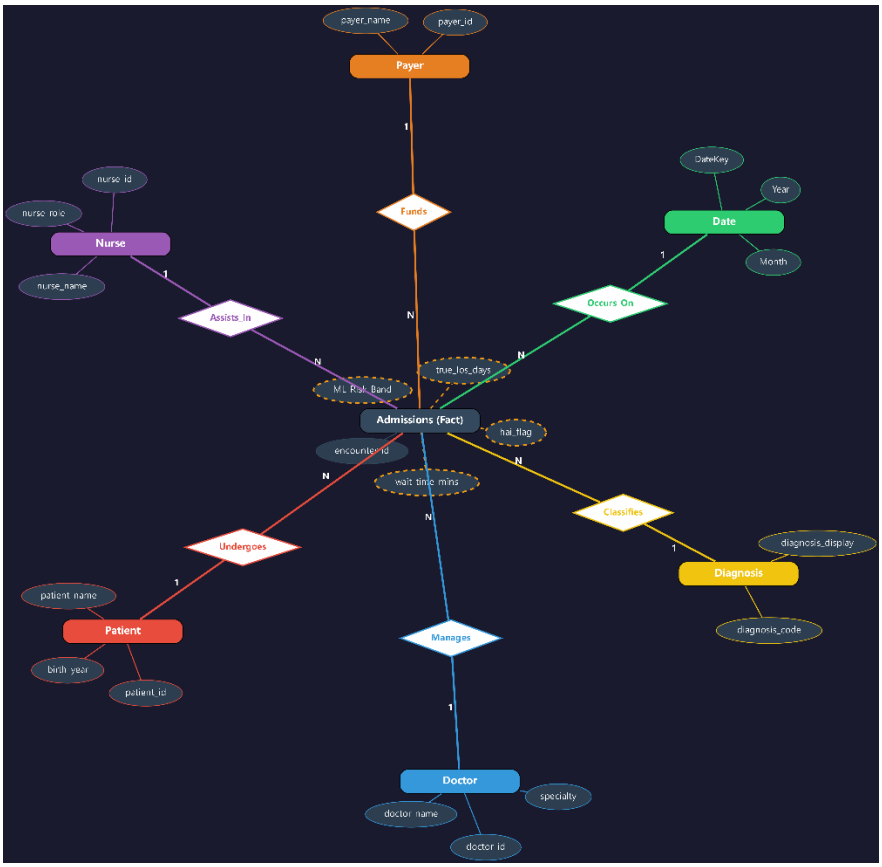
- **Feedback Loop:** Uploads predictions (ML_safety_incident_flag, ML_safety_risk_score, ML_predicted_los_days, ML_predicted_total_cost) back to the SQL database (silver.ML_Results) via SQLAlchemy's fast_executemany for downstream BI consumption.

6. Power BI Semantic Model (BI Layer)

The finalized Gold schema feeds directly into a robust Power BI Semantic Model, structured using Tabular Model Definition Language (TMDL).

6.1 Star Schema Relationships (relationships.tmdl)

- Fact_Admissions acts as the central fact table.
- One-to-Many physical relationships connect to Dim_Diagnosis, Dim_Doctor, Dim_Nurse, Dim_Patient, Dim_Payer, and Dim_Date.
- Multiple inactive/active date relationships are mapped for role-playing dates (Triage, Assessment, Bed Request, Admit, Discharge).



Figer 1 : Healthcare Conceptual ERD

6.2 Advanced DAX KPIs (KPIs Measure.tmdl)

Metrics are rigorously organized into distinct Display Folders for user accessibility:

- **Executive KPIs:** Total Admissions, Avg Length of Stay, Total Expected Revenue, High Satisfaction %.
- **Clinical & Safety KPIs:** HAI Rate %, 30-Day Readmission Rate %, Hypertensive Crisis Rate %, Safety Incident Rate %, Avg Medication Adherence, Diabetic Poor Control Rate %, Low Medication Adherence %, Avg Severity Index, Complex Patients Rate %.
- **Patient Experience KPIs:** Avg Doctor Communication Score, Avg Nurse Communication Score, Avg Cleanliness Score, Telehealth Adoption Rate %, 48h Follow-up Rate %.
- **Operational KPIs:** Avg Actual Wait Time, Avg Perceived Wait Time, SLA Breach Rate, Avg Bed Lag Time (Mins), Min/Max Length of Stay, Bed Occupancy Rate %.
- **Financial KPIs:** Total Submitted Charges, Average Write-off %, Claim Denial Rate %, Net/Absolute Billing Errors, Gross Charges, Insurance Revenue, Patient Revenue, Revenue Leakage, Insurance Leakage.
- **HR KPIs:** Total Overtime Hours, Overloaded Shifts %, High Burnout Rate, Critical Turnover Risk %, Total Active Nurses, Avg Patients Per Nurse.
- **Machine Learning (ML KPIs):** Avg LOS Variance, ML MAE (LOS), Predicted Incident Rate, Critical Risk Patients, Overstay Ratio %, Profit Prediction Gap.

6.3 Time Intelligence (Time Intelligence.tmdl)

- Implements a **Calculation Group** allowing dynamic time-based analysis across any measure in the model.
- Provides instant toggles for: Year-to-Date (YTD), Month-to-Date (MTD), Quarter-to-Date (QTD), Previous Year (PY), Previous Month (PM), Year-over-Year % (YoY %), and Month-over-Month % (MoM %).

7. Error Handling and Resilience

Across all SQL pipeline procedures and Python scripts, strict TRY...CATCH blocks and try/except handlers are implemented:

1. Error messages are captured via SQL's ERROR_MESSAGE() or Python's exceptions.
2. Failed tasks and jobs are immediately logged as Failed in the ops SQL logging tables and the local pipeline_orchestrator.log.
3. In SQL, errors are re-thrown (THROW;) to alert the orchestrator, which safely exits the sequence without corrupting downstream layers.

8. End-to-End Deployment / Execution Order

To deploy and execute this data warehouse from scratch, run the scripts in the following sequence:

1. **SQL Initialization (One-time setup):** * database_definetion.sql -> logging_utilites.sql
    - Creat Staging Layer.sql -> ddl_bronze.sql -> ddl_SilverLayer.sql -> ddl_goldLayer.sql
2. **System Orchestration (Daily operations):**
    - Start scheduler.py. This script automatically manages:
        1. Data Generation
        2. Database Import
        3. Medallion ETL Transformation (Python run loading code.py)
3. **Machine Learning Execution (Optional / Scheduled):**
    - Run ML3.py to generate new predictions based on the latest Silver layer data.
4. **BI Layer Refresh:**
    - Refresh the Power BI Semantic Model to ingest the latest facts, dimensions, ML scores, and update calculation groups.